

# PointsTrack

## System Architecture, Robustness & Scalability

*A technical report — how the platform is built, why it stays correct under concurrent load, and how it scales.*

**Stack:** React Native + Expo · Next.js · Express + TypeScript · PostgreSQL + Drizzle ORM · JWT

**Evidence:** *every guarantee in this report is backed by the live codebase and by automated tests executed against a running server and a real PostgreSQL database.*

**Generated:** 2026-06-14

# 1. What PointsTrack is

AICTE requires engineering students to earn 100 activity points (80 for lateral entry) to graduate. Most colleges still track this manually in spreadsheets and PDFs, which is error-prone, unverifiable and easy to lose. PointsTrack digitizes the whole loop:

- **Organizers** publish events, set capacity, and check students in — manually or by scanning a QR code.
- **Students** discover events, register, get checked in, and watch points land in their wallet toward their goal.
- **The platform** awards points automatically and exactly once, tracks progress, and notifies both sides.

It replaces a previous Firebase stack with a custom, self-hosted backend — for full control over data, cost, and correctness under concurrent load (the dimension a managed BaaS makes hardest to reason about).

# 2. System architecture

Three components share a single stateless API. The clients never talk to the database or to each other; everything goes through the API over HTTPS with a JWT bearer token.

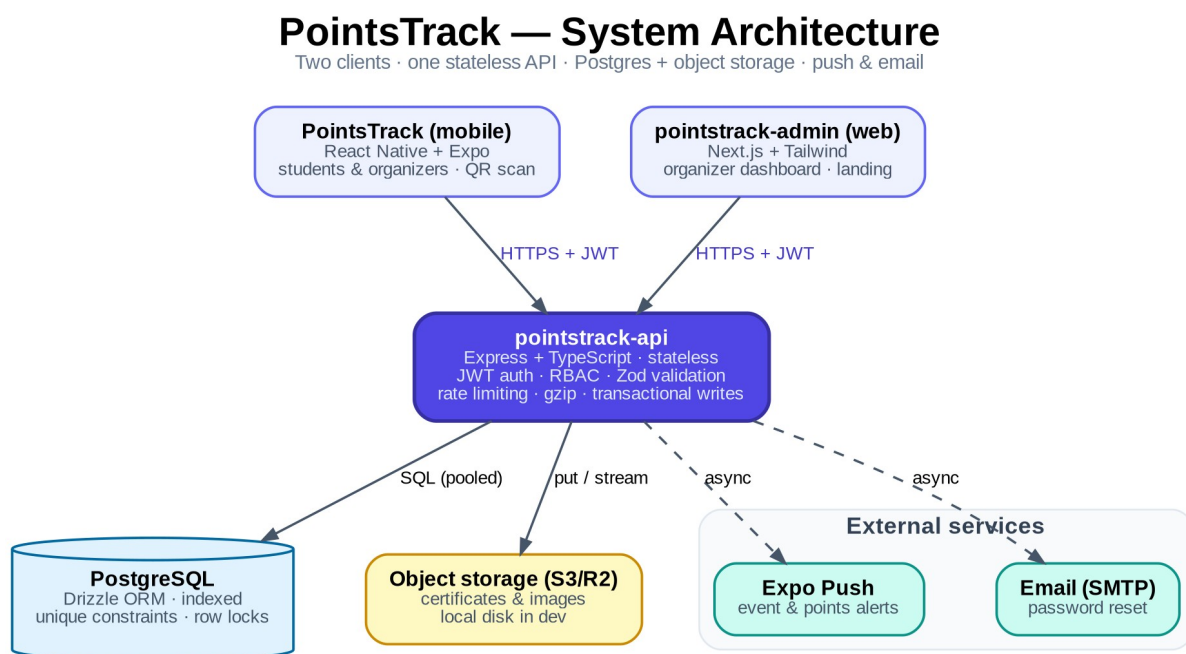


Figure 1 — System architecture: two clients, one stateless API, Postgres + object storage, async push & email.

| Component         | Technology                                             | Who uses it                  |
|-------------------|--------------------------------------------------------|------------------------------|
| pointstrack-api   | Express + TypeScript,<br>PostgreSQL + Drizzle ORM, JWT | Backend for both clients     |
| pointstrack-admin | Next.js (App Router) + Tailwind                        | Organizers (web dashboard) + |

|             |                                                 |                                |
|-------------|-------------------------------------------------|--------------------------------|
|             | + Framer Motion                                 | public landing                 |
| PointsTrack | React Native + Expo,<br>NativeWind, expo-camera | Students + organizers (mobile) |

### 3. Data model

PostgreSQL via Drizzle ORM. One identity table (accounts) feeds two profile tables; events, attendance and the points “wallet” are separate tables linked by foreign keys. Every foreign key cascades on delete, so removing an account cleanly removes all of its data.

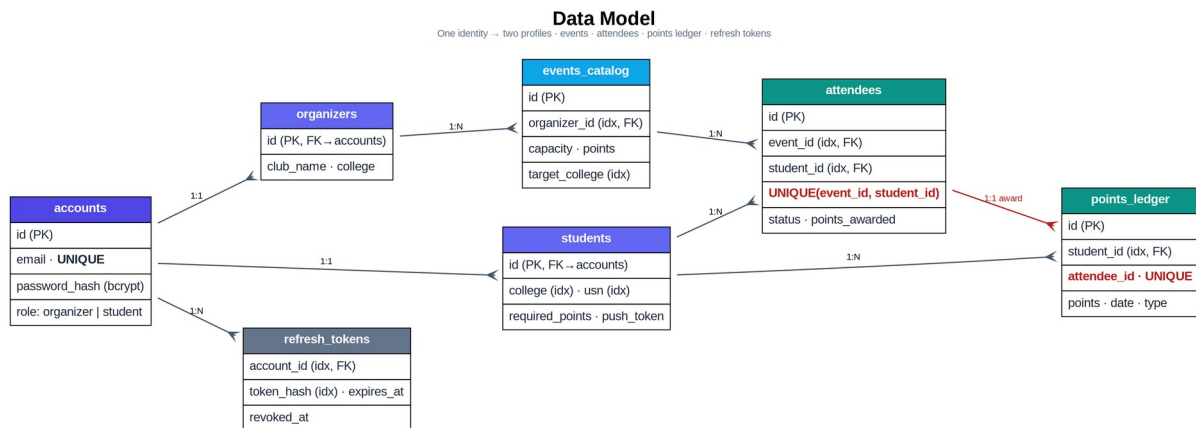


Figure 2 — Data model. The two red constraints carry the system's correctness guarantees.

Two unique constraints do the heavy lifting for correctness:

- **UNIQUE(event\_id, student\_id)** on attendees — a student can register for an event at most once.
- **UNIQUE(attendee\_id)** on points\_ledger — a single check-in can award points exactly once.

These are enforced by the database itself, so they hold even if application logic has a bug or two requests race. The schema also indexes every hot lookup (organizer, event, student, college, email, token hash).

### 4. The check-in flow

The core transaction is the check-in that awards points. End to end:

1. An organizer creates an event; eligible students are notified via Expo Push.
2. A student sees it in their feed and registers. If the event is full, they are waitlisted instead.
3. At the venue the organizer checks the student in — tapping “Allot Points”, or scanning their QR code.
4. Inside a single DB transaction the API locks the attendee row, flips status to checked-in, and writes exactly one ledger row.
5. Points appear instantly in the student's wallet and count toward their goal.

6. If a checked-in student is later rejected, the freed seat auto-promotes the longest-waiting waitlisted student.

## 5. Robustness — correct under concurrency

The genuinely hard part of an attendance/points system is concurrent writes: two people tapping “check in” at once, a double-tapped button, or a popular event filling up as dozens register simultaneously. Naïve code corrupts data here — double points, over-capacity, duplicate rows. PointsTrack defends at two layers.

### 5.1 Two layers of defense

#### Primary defense — transactional logic with row locks.

Point awards, capacity checks, waitlist promotion and QR walk-up creation all run inside `db.transaction` with `SELECT ... FOR UPDATE` row locks. The capacity check locks the event row so concurrent applicants are serialised and seats can't be over-sold; the award locks the attendee row and re-checks status inside the lock so only the first check-in awards points. Lock order is consistent (event row before attendee rows) to avoid deadlocks, and side effects (push, waitlist promotion) run only after the transaction commits — never holding a lock open over network I/O.

```
// attendees.ts — the award, inside the transaction
const [attendee] = await tx.select().from(attendees)
  .where(eq(attendees.id, attendeeId)).for('update'); // lock the row
...
await tx.insert(pointsLedger).values({ attendeeId: attendee.id, ... })
  .onConflictDoNothing({ target: pointsLedger.attendeeId }); // award once
```

#### Backstop — database constraints.

Even if the application logic were wrong, the unique indexes make corruption physically impossible: a duplicate registration or a second award row is rejected by Postgres.

## Concurrency-Safe Check-in → Points Award

Row lock + unique index = points awarded exactly once, even under a storm of double-taps

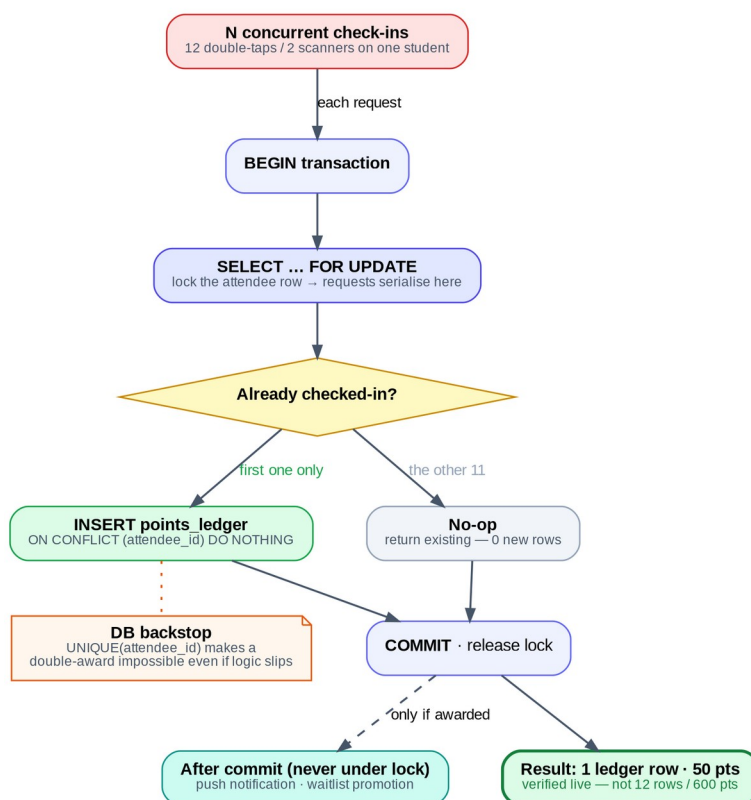


Figure 3 — Concurrency-safe check-in. Of  $N$  simultaneous requests, the row lock lets exactly one through to award points; the unique index is the backstop.

## 5.2 Proof — live concurrency tests

These guarantees are not asserted — they are demonstrated. A test harness fires concurrent HTTP requests at the running API (the same routes, transactions and constraints used in production), then reads ground truth directly from PostgreSQL. Nothing is mocked.

| Concurrency scenario                      | Expected                | Actual (measured live)         |
|-------------------------------------------|-------------------------|--------------------------------|
| 12 simultaneous check-ins of one attendee | 1 ledger row · 50 pts   | <b>1 ledger row · 50 pts</b>   |
| 10 simultaneous duplicate applications    | 1 attendee row          | <b>1 row (9 × HTTP 409)</b>    |
| capacity = 1, 6 simultaneous applications | 1 active · 5 waitlisted | <b>1 active · 5 waitlisted</b> |

**Result: 3 / 3 scenarios correct under concurrency — ALL PASS.** Reproducible with `npx tsx scripts/concurrency-proof.ts` against `localhost:4000` + PostgreSQL 16 (run 2026-06-14).

Without the row lock + unique index, scenario 1 alone would have written 12 ledger rows and awarded 600 points instead of 50 — the classic double-award bug. The test asserts the bug cannot occur.

### 5.3 Error handling

Error handling is centralized and leak-free: Zod validation failures → 400, typed HttpErrors → their status, Postgres unique violations → 409, and anything unexpected → 500 with a clean message and no stack trace exposed to the client.

## 6. Scalability — built to grow

Scalability here means two things: the architecture can add capacity without a rewrite, and the current single instance already performs well. Both are addressed deliberately.

### 6.1 Design choices that enable scale

- **Stateless API.** Authentication is a JWT carried in each request; the server keeps no session state. You can therefore run N identical instances behind a load balancer and scale horizontally.
- **Bounded queries.** Every list endpoint uses capped limit/offset pagination, so no request can ever ask the database for an unbounded number of rows — payloads and memory stay flat as data grows.
- **Efficient reads.** Indexes cover every hot lookup; per-event registration counts come from a single grouped query rather than one query per event.
- **Swap-in managed database.** The app touches only DATABASE\_URL, so dropping in managed Postgres (Neon/Supabase) for autoscaling, replicas and branching is a config change. Pooling is configurable via DB\_POOL\_MAX, with pgBouncer/Neon pooling for many instances.
- **Storage abstraction.** Uploads go to S3-compatible object storage (Cloudflare R2) in production and local disk in dev, switched by env vars with zero code change. This is precisely what lets the API run as multiple stateless instances.
- **Lean transport.** Responses are gzip-compressed; push notifications are chunked through the Expo SDK.

## 6.2 Scaling topology

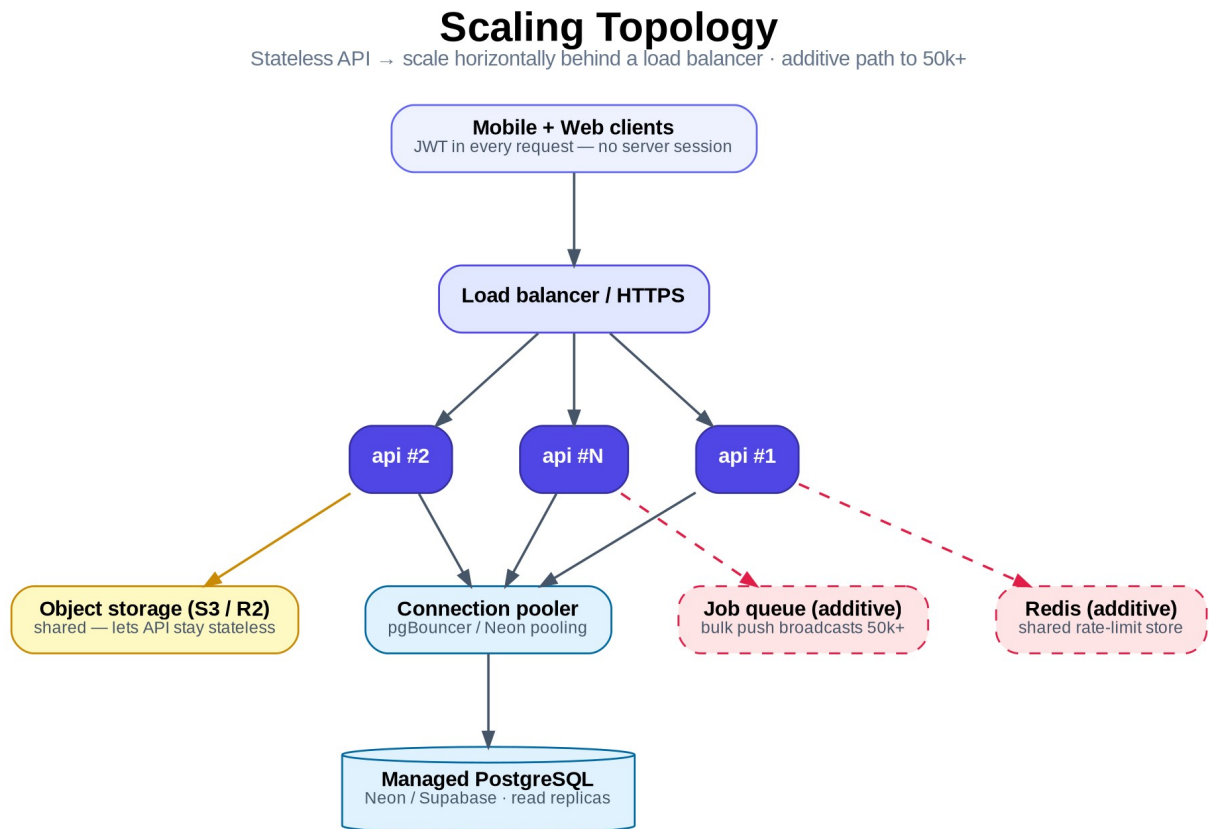


Figure 4 — Scaling topology. Stateless instances scale out behind a load balancer; the growth path (Redis, job queue) is additive.

## 6.3 A real measurement

To put a number on it, a single un-tuned development instance (with verbose request logging on) against a local PostgreSQL was driven with a read load test:

| Metric                    | Value                                                 |
|---------------------------|-------------------------------------------------------|
| Endpoint                  | GET /events (paginated, indexed, authenticated, gzip) |
| Requests / concurrency    | 2,000 requests @ 50 concurrent                        |
| Throughput                | 358 requests / second                                 |
| Latency — p50 / p95 / p99 | 134 ms / 180 ms / 246 ms                              |

*This is a deliberately conservative figure — a single small instance, no production tuning, dev-mode logging enabled. The point is the headroom: throughput scales roughly linearly by adding stateless instances, and read-heavy traffic can be offloaded to Postgres read replicas.*

## 6.4 Honest capacity & roadmap

- **As-is (one API instance + managed Postgres):** comfortably a full college — low-thousands of users — and correct under bursty check-in load, which is the moment most apps break.

- **To 50k+ / multi-college:** add a Redis-backed rate-limit store, a background job/queue for very large push broadcasts, and object storage. All additive — roughly a day of work, no architectural change.

## 7. Security

- **JWT + rotating refresh tokens.** Short-lived access tokens (~15 min); refresh tokens are hashed in the DB with expiry and revocation, and rotated on every use. Stolen refresh tokens can be revoked; access tokens expire fast.
- **Passwords hashed with bcrypt.** No plaintext credentials are ever stored.
- **Role-based access + ownership checks.** `requireRole` guards every protected route; an organizer can only touch their own events and attendees.
- **Rate limiting.** A generous global ceiling on every route, plus a strict limiter on `/auth` to blunt brute-force and credential-stuffing.
- **Hardened transport.** helmet security headers, a strict CORS allow-list in production, and a 1 MB JSON body limit.

## 8. Conclusion

PointsTrack is built so that correctness is enforced by the database rather than hoped for in application code: row locks serialise the races, and unique constraints make corruption impossible. That correctness is demonstrated, not merely claimed — three concurrency scenarios that break naïve implementations all pass when run live against a real server and database. The API is stateless by design, so it scales horizontally with no architectural change, and the path to 50k+ users is additive. A single un-tuned instance already sustains ~358 req/s on paginated reads. In short: robust where it counts, and ready to grow.

**Appendix — reproduce the proofs:** `npx tsx scripts/concurrency-proof.ts` (correctness) · `npx tsx scripts/load-proof.ts` (throughput)